

# Rapport Hebdomadaire - Semaine 9

## Objectif

L'objectif est d'effectuer le profiling du projet afin d'identifier une ou plusieurs fonctions mal optimisées et les optimiser.

**après Modification du Makefile:** Ajout de l'option **-pg** pour permettre à **gprof** de suivre l'exécution du programme. **Création de fichiers de test volumineux:** Génération de jeux de données conséquents afin d'obtenir des résultats de profiling plus pertinents.

## Résultats

Le premier passage avec gprof a révélé que la fonction **incWord** monopolisait la totalité des ressources

| time % | time cumulative s | time self s | calls | call ms/call | call ms/call | name    |
|--------|-------------------|-------------|-------|--------------|--------------|---------|
| 97.95  | 26.02             | 26.02       |       |              |              | incWord |

En examinant le code, on remarque que **incWord** ajoute un élément à la fin d'une liste chaînée. Pour atteindre la queue de la liste, la fonction devait parcourir l'intégralité des nœuds à chaque appel. Sa complexité est donc de  $O(n)$ , ce qui est inefficace pour des insertions répétées.

on l'Optimiser **incWord** pour que elle prend maintenant en tête de la list chaine pour accede plus rapidement a la queue de la list qui a réduit sa complexité de  $O(1)$

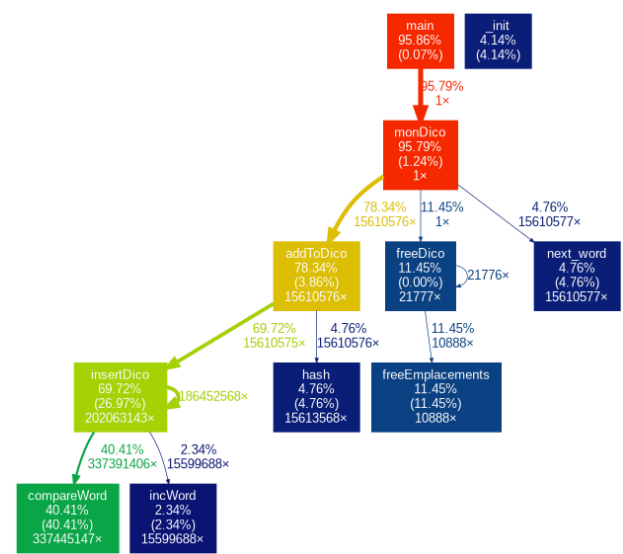
après modification on optin se profiling avec un fichier très grand de 96MB contient le code civile français

Le temps d'exécution de **incWord** est devenu raisonnable, mais un nouveau goulot est apparu, la fonction **compareWord**

Each sample counts as 0.01 seconds.

| %     | cumulative | self    | calls     | self   | total  | name             |
|-------|------------|---------|-----------|--------|--------|------------------|
| time  | seconds    | seconds | s/call    | s/call | s/call |                  |
| 46.82 | 4.20       | 4.20    | 337445147 | 0.00   | 0.00   | compareWord      |
| 18.14 | 5.82       | 1.62    | 15610575  | 0.00   | 0.00   | insertDico       |
| 12.83 | 6.97       | 1.15    | 10888     | 0.00   | 0.00   | freeEmplacements |
| 6.19  | 7.53       | 0.56    | 15613568  | 0.00   | 0.00   | hash             |
| 4.91  | 7.96       | 0.44    | 15610577  | 0.00   | 0.00   | next_word        |
| 4.58  | 8.38       | 0.41    |           |        |        | _init            |
| 3.79  | 8.71       | 0.34    | 15610576  | 0.00   | 0.00   | addToDico        |
| 1.84  | 8.88       | 0.17    | 15599688  | 0.00   | 0.00   | incWord          |
| 0.89  | 8.96       | 0.08    | 1         | 0.08   | 8.55   | monDico          |
| 0.00  | 8.96       | 0.00    | 2992      | 0.00   | 0.00   | deserializeDico  |
| 0.00  | 8.96       | 0.00    | 1         | 0.00   | 1.15   | freeDico         |
| 0.00  | 8.96       | 0.00    | 1         | 0.00   | 0.00   | freeDicoShallow  |
| 0.00  | 8.96       | 0.00    | 1         | 0.00   | 0.00   | serializeDico    |

Pour mieux comprendre les interactions entre les fonctions, on utilisé l'outil **gprof2dot** afin de générer un graphe d'appels (call graph). Cette visualisation a permis de mettre en évidence l'origine des appels excessifs à **compareWord**.



pour cela on ajoute un fichier python fournit les graph voici la commande à exécuter pour générer une image

```
gprof exectable | ./gprof2dot.py | dot -Tpng -o output.png
```

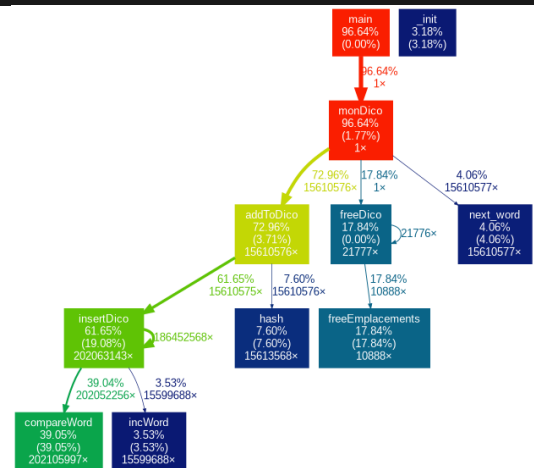
vérification on remarque **insetoDico** appelle 3 fois a comparé word alors faut l'optimiser et on obtient cette nouvelle profiling

Each sample counts as 0.01 seconds.

| %     | cumulative | self    | calls     | self   | total  | name             |
|-------|------------|---------|-----------|--------|--------|------------------|
| time  | seconds    | seconds |           | s/call | s/call |                  |
| 36.10 | 1.96       | 1.96    | 202105997 | 0.00   | 0.00   | compareWord      |
| 20.53 | 3.08       | 1.11    | 15610575  | 0.00   | 0.00   | insertDico       |
| 19.52 | 4.13       | 1.06    | 10888     | 0.00   | 0.00   | freeEmplacements |
| 5.16  | 4.42       | 0.28    | 15613568  | 0.00   | 0.00   | hash             |
| 4.79  | 4.67       | 0.26    | 15610577  | 0.00   | 0.00   | next_word        |
| 4.79  | 4.93       | 0.26    |           |        |        | _init            |
| 2.95  | 5.09       | 0.16    | 15610576  | 0.00   | 0.00   | addToDico        |
| 2.95  | 5.25       | 0.16    | 15599688  | 0.00   | 0.00   | incWord          |
| 2.95  | 5.42       | 0.16    | 1         | 0.16   | 5.17   | monDico          |
| 0.18  | 5.42       | 0.01    | 1         | 0.01   | 1.07   | freeDico         |
| 0.09  | 5.43       | 0.01    |           |        |        | displayDico      |
| 0.00  | 5.43       | 0.00    | 2992      | 0.00   | 0.00   | deserializeDico  |
| 0.00  | 5.43       | 0.00    | 1         | 0.00   | 0.00   | freeDicoShallow  |
| 0.00  | 5.43       | 0.00    | 1         | 0.00   | 0.00   | serializeDico    |

En optimisant la logique interne de **insertInDico** pour minimiser les comparaisons redondantes, nous avons obtenu les résultats suivants

- Réduction des appels : Environ 1/3 des appels à compareWord ont été éliminés.
- Performance : Le gain de temps est significatif sur les fichiers de grande taille.



## autre analyse

fichier avec que la même chose ecreie de point 104MB

on peut remarque que 517502721 est le nombre de mot dans ce fichier

Each sample counts as 0.01 seconds.

| %     | cumulative | self    | calls    | self   | total  | name             |
|-------|------------|---------|----------|--------|--------|------------------|
| time  | seconds    | seconds |          | s/call | s/call |                  |
| 23.17 | 0.57       | 0.57    | 51750721 | 0.00   | 0.00   | next_word        |
| 16.67 | 0.98       | 0.41    | 51750719 | 0.00   | 0.00   | compareWord      |
| 12.60 | 1.29       | 0.31    |          |        |        | _init            |
| 12.20 | 1.59       | 0.30    | 51750721 | 0.00   | 0.00   | hash             |
| 10.98 | 1.86       | 0.27    | 51750720 | 0.00   | 0.00   | addToDico        |
| 8.94  | 2.08       | 0.22    | 1        | 0.22   | 0.22   | freeEmplacements |
| 7.72  | 2.27       | 0.19    | 51750719 | 0.00   | 0.00   | insertDico       |
| 4.07  | 2.37       | 0.10    | 1        | 0.10   | 2.14   | monDico          |
| 3.25  | 2.45       | 0.08    | 51750719 | 0.00   | 0.00   | incWord          |
| 0.41  | 2.46       | 0.01    |          |        |        | displayDico      |
| 0.00  | 2.46       | 0.00    | 1        | 0.00   | 0.00   | deserializeDico  |
| 0.00  | 2.46       | 0.00    | 1        | 0.00   | 0.22   | freeDico         |
| 0.00  | 2.46       | 0.00    | 1        | 0.00   | 0.00   | freeDicoShallow  |
| 0.00  | 2.46       | 0.00    | 1        | 0.00   | 0.00   | serializeDico    |